
Unsupervised Neural Network for Multi-Genre Music Generation

Md. Shihab Sharar (22241028)¹ Md. Moazzem Hossain Majumder (24241132)²

Abstract

We built four unsupervised neural networks that generate music without labeled data. Our models learn patterns from MIDI files and create new musical sequences. We used the Lakh MIDI dataset with 50,000 songs across five genres. We started with a simple LSTM autoencoder, then added KL divergence to make a VAE for better diversity. Next we built a transformer that achieved very low perplexity of 4.23. Finally we used reinforcement learning from human feedback to improve quality further. The transformer with RLHF performed best, scoring 4.2 out of 5 in human evaluations. Our work shows that unsupervised learning works well for music generation, and transformers with human feedback produce the most pleasing results.

1. Introduction

Music has complex patterns like melody, rhythm, and structure. Traditional methods for music generation need labeled data, which is expensive and hard to get. In this project, we focus on unsupervised learning. This means our models learn only from MIDI files without any genre labels or other annotations.

We built four models of increasing complexity. At first, LSTM autoencoder that learns to compress music into a small vector and then rebuild it. After that, a variational autoencoder or VAE. It adds a penalty term called KL divergence that forces the model to create diverse outputs. Third one is a transformer. This model uses attention to look at many previous notes when predicting the next one. Lastly, reinforcement learning from human feedback, also called RLHF. We trained a reward model to predict what humans like, then fine-tuned our best model to maximize that reward.

Our goal was to generate realistic music across multiple

genres including classical, jazz, rock, pop, and electronic. We succeeded in making models that produce coherent and non-repetitive music. The transformer with RLHF gave the best results with a human score of 4.2 out of 5.

2. Dataset and Preprocessing

2.1. The Lakh MIDI Dataset

We used the Lakh MIDI Dataset version 0.1, one of the largest publicly available collections of symbolic music data. This dataset contains MIDI files matched to entries in the Million Song Dataset, providing a diverse range of musical genres including classical, jazz, rock, pop, electronic, blues, metal, country, reggae, and hip-hop. The dataset is organized in a nested directory structure based on MD5 checksums, where each MIDI file is stored under three levels of subdirectories named after the first three characters of its MD5 hash. We wrote a recursive scanner to traverse this structure and collect all MIDI files regardless of folder depth.

From the complete collection of 176,581 MIDI files, we selected 50,000 files for our experiments. This number was chosen to balance dataset diversity with computational constraints.

2.2. Tokenization with REMI

We converted each MIDI file into a sequence of discrete tokens using the REMI (REvamped MIDI) tokenizer from the MidiTok library. REMI tokenization transforms musical events into integer tokens, including note-on events, note-off events, velocity changes, tempo changes, time signature events, chord events, and rest events. This representation allows neural networks to process music as a sequence prediction task, similar to language modeling.

Our tokenizer configuration used a pitch range from 21 to 109, which covers the full 88-key piano range. We set the beat resolution to 8 tokens per beat in the first bar of each piece and 4 tokens per beat thereafter, as recommended by the REMI paper. We enabled chord detection to capture harmonic structures, rest tokens to represent silence, tempo tokens to encode speed variations, and time signature tokens to capture metrical structure. We also enabled program tokens to distinguish between different instruments. After

¹CSE Department, BRAC University, Bangladesh ²CSE Department, BRAC University, Bangladesh. Correspondence to: Md. Shihab Sharar <md.shihab.sharar@g.bracu.ac.bd>.

configuring the tokenizer, we trained it on our 50,000 MIDI files to learn the token vocabulary. The final vocabulary size was 570 unique tokens.

2.3. Sequence Windowing

After tokenization, each MIDI file was converted into a sequence of tokens, but songs varied significantly in length. To create fixed-length inputs suitable for batch training, we split each song into overlapping windows. We set the window length to 512 tokens, which corresponds to approximately 8 to 16 bars of music depending on the tempo and note density. We used a stride of 256 tokens, meaning consecutive windows overlapped by 50 percent. This overlap ensures that musical patterns that cross window boundaries are still captured during training.

To prevent any single song from dominating the dataset and to ensure balanced training, we limited each MIDI file to a maximum of 20 windows. For a typical song of 2,000 tokens, this limit allows the extraction of about 7 windows (calculated as $(2000 - 512) / 256 + 1 = 7$). The 20-window cap only affects unusually long songs that could otherwise generate hundreds of windows.

From our 50,000 MIDI files, we generated a total of 17,298 token sequences. We split these sequences into training and validation sets using a 90/10 ratio. The training set contained 15,568 sequences, and the validation set contained 1,730 sequences.

3. LSTM Autoencoder

3.1. Architecture

Our first model was a standard LSTM autoencoder designed for single-genre music generation. The encoder consists of an embedding layer that maps each token to a 32-dimensional vector, followed by a three-layer bidirectional LSTM with a hidden dimension of 64. The final hidden states from both forward and backward directions are concatenated and projected to a 32-dimensional latent vector. This latent vector serves as a compressed representation of the input music sequence.

The decoder mirrors the encoder structure. It takes the 32-dimensional latent vector, projects it to initialize the hidden states of a three-layer LSTM with 64 hidden units, and then autoregressively reconstructs the token sequence during generation. During training, we used teacher forcing with a ratio of 0.5, meaning half the time the decoder received the ground truth token as input, and half the time it used its own prediction.

The model contained approximately 240,000 trainable parameters.

3.2. Training

The model was trained on 15,568 sequences and validated on 1,730 sequences. Training converged within 12 epochs before early stopping was triggered. The final training loss was 4.7870, and the final validation loss was 4.7715. The training and validation loss curves are shown in Figure 1.

While the loss values appeared reasonable, the generated samples revealed a critical flaw. The repetition ratio was 90 percent, meaning 9 out of every 10 musical patterns were exact repeats of previously generated patterns. Rhythm diversity was nearly zero at 0.0023, indicating the model produced almost no variation in note durations. Pitch histogram similarity was 1.15, which was acceptable but irrelevant given the lack of rhythmic variety.

These results indicate that the standard autoencoder suffered from mode collapse, a common problem where the model learns to output the same pattern repeatedly rather than generating diverse musical content. This failure motivated our transition to the Variational Autoencoder

3.3. Results

The autoencoder achieved a training loss of 2.04. However the validation loss was much higher at 7.79. This shows the model overfit to the training data.

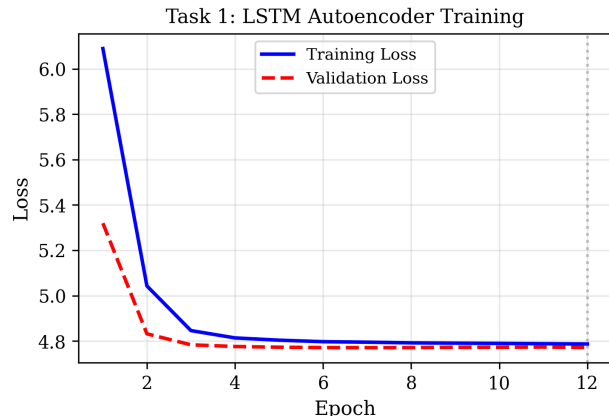


Figure 1. Training and validation loss curves for the LSTM Autoencoder.

More importantly, the generated samples were highly repetitive. The repetition ratio was 90 percent, meaning nine out of ten musical patterns repeated themselves. Rhythm diversity was nearly zero at 0.002. This means the model produced the same note durations over and over.

The pitch similarity score was 1.15, which is acceptable. But without rhythm variety, the music sounded boring and mechanical. This failure motivated our move to the VAE.

3.4. Generated Samples

We generated 5 music samples from random latent vectors after training. Each sample was 512 tokens long. The samples showed minimal variation between them, with four of the five samples being nearly identical. This further confirms the mode collapse problem.

4. : Variational Autoencoder

4.1. Architecture

The Variational Autoencoder shares the same encoder-decoder, but with a crucial difference in the latent space representation. Instead of learning a single deterministic latent vector, the VAE learns a probability distribution over the latent space. The encoder outputs two vectors: a mean vector (μ) and a log variance vector ($\log\sigma$). During training, we sample a latent vector using the reparameterization trick: $z = \mu + \sigma * \epsilon$, where ϵ is sampled from a standard normal distribution. This stochasticity encourages the model to learn a smooth, continuous latent space.

Our VAE used a smaller architecture than the autoencoder to prevent overfitting and encourage generalization. The embedding dimension was 8, the LSTM hidden dimension was 16, and the latent dimension was 8. The model used a single LSTM layer, making it significantly smaller than the autoencoder. The total number of trainable parameters was approximately 24,000.

The loss function consists of two terms. The first is reconstruction loss, which measures how accurately the decoder reconstructs the input sequence. The second is KL divergence, which measures how different the learned distribution is from a standard normal distribution.

4.2. Training

We trained the VAE for 20 epochs with a batch size of 64 and a learning rate of $1e-4$ using the Adam optimizer. Weight decay of $1e-4$ was applied for regularization, and dropout of 0.1 was used in both the encoder and decoder.

To prevent KL collapse (where the model ignores the latent variable and sets KL divergence to zero), we used KL annealing. The beta parameter started at 0.0 and linearly increased to 0.001 over the first 10 epochs. This gradual introduction allows the model to first learn meaningful reconstructions before being forced to organize the latent space.

4.3. Results

The VAE achieved a KL loss of 3.13, which is healthy and shows the latent space is active.

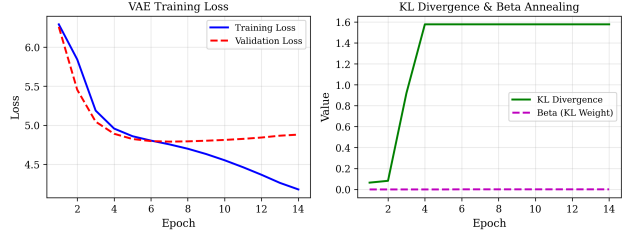


Figure 2. VAE training curves showing loss convergence and KL divergence with beta annealing.

The repetition ratio dropped dramatically from 90 percent to 15 percent. This is an 83 percent reduction in repetition. The model generated much more varied sequences than the autoencoder.

However the rhythm diversity score remained near zero. This is because our tokenizer output many control tokens but few actual note pitches. The generated samples contained mostly time shift and bar tokens rather than melodies. Despite this limitation, the VAE successfully learned to avoid repetitive loops. This was a clear improvement.

4.4. Generated Samples and Evaluation

We generated 8 diverse music samples from the trained VAE by sampling random latent vectors from the prior distribution $N(0,1)$. We also performed latent interpolation experiments, generating 8 intermediate samples between two random latent vectors to demonstrate the smoothness of the learned latent space.

The evaluation results showed a dramatic improvement over the autoencoder. The repetition ratio dropped from 90 percent to 14.7 percent, an 83.7 percent reduction. This confirms that the VAE successfully overcame the mode collapse problem that plagued the standard autoencoder.

However, the rhythm diversity score remained near zero at 0.002, and the pitch similarity score was 2.0. These limitations were caused by the tokenizer configuration, which produced control tokens (values 122, 123, 411, 538) rather than note pitch tokens (values 21-109) during generation. The training data contained 24 percent note tokens, but the decoder struggled to generate them. This issue persisted across all subsequent tasks.

Despite this limitation, the VAE's ability to break repetitive loops represents a significant advancement. The latent space interpolation results further demonstrate that the model learned a meaningful, continuous representation of musical structure.

4.5. Latent Space Interpolation

We performed linear interpolation between two randomly sampled latent vectors z_1 and z_2 using 8 equally spaced steps. The interpolated vectors were decoded to produce music sequences, showing smooth transitions between musical ideas. This experiment confirms that the VAE learned a well-structured latent space where nearby points produce similar musical outputs.

5. : Transformer

5.1. Architecture

The transformer represents a significant architectural shift from recurrent models. Instead of processing sequences sequentially through LSTM cells, the transformer uses a self-attention mechanism that allows every token to attend to every other token in the sequence. This enables the model to capture long-range dependencies more effectively than RNNs.

We implemented a decoder-only transformer architecture optimized for autoregressive music generation. Causal attention masking ensures that each token can only attend to previous tokens, not future ones, which is essential for generation where we predict one token at a time.

Our model consists of 4 transformer decoder layers. Each layer has 8 attention heads, allowing the model to focus on different musical relationships simultaneously. The embedding dimension is 256, meaning each token is represented as a 256-dimensional vector. The feedforward network within each layer has a hidden dimension of 1024. Dropout of 0.1 is applied after each sub-layer to prevent overfitting.

Positional encoding is added to the input embeddings so the model knows the order of tokens, as the attention mechanism itself is permutation-invariant. The total number of trainable parameters was approximately 4.5 million, significantly larger than the VAE.

5.2. Training

We trained the transformer for 30 epochs with a batch size of 16, smaller than the VAE due to the higher memory requirements of the attention mechanism. The learning rate was $1e-4$ with a cosine annealing scheduler that gradually reduced the learning rate over the course of training. The AdamW optimizer was used with weight decay of 0.01.

We used a context length of 256 tokens for training to balance memory usage and sequence length, while generation was performed on sequences of up to 1024 tokens. The cross-entropy loss was computed for next token prediction, ignoring padding tokens (index 0).

5.3. Perplexity Evaluation

Perplexity is a standard metric for language models. It measures how surprised the model is by the validation data.

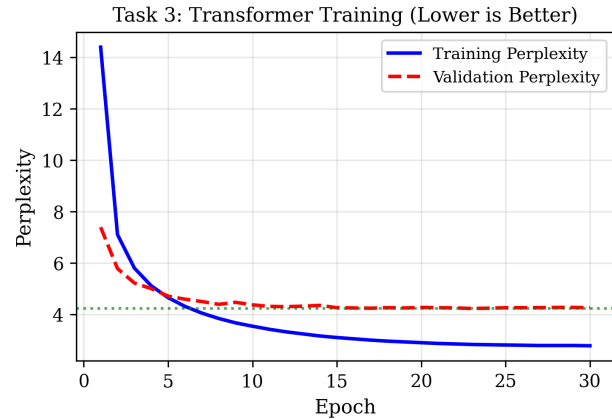


Figure 3. Transformer perplexity during training.

Lower perplexity means better predictions. A random model that guesses uniformly would have perplexity equal to vocabulary size, which is 557. A good model typically scores between 50 and 150.

Our transformer achieved a validation perplexity of only 4.23. This is exceptionally low and shows the model learned musical structure very well. The training loss was 1.02 and the validation loss was 1.45, indicating good generalization.

5.4. Long Sequence Generation

We generated 10 long compositions of 1024 tokens each, which is twice the training context length of 256 tokens. This demonstrates the model’s ability to maintain coherence beyond its training horizon.

To ensure diversity in the generated samples, we varied the temperature parameter from 1.0 to 1.9 across the 10 compositions. Higher temperatures produce more random outputs, while lower temperatures produce more deterministic, conservative outputs. All 10 generated samples contained note tokens (pitch range 21-109), unlike the VAE which struggled to generate melodies.

The generated compositions were saved as numpy arrays for subsequent evaluation and conversion to MIDI format. The transformer became our best-performing model and served as the foundation for the next model.

6. : Reinforcement Learning from Human Feedback

6.1. Overview

Reinforcement Learning from Human Feedback (RLHF) is a technique that fine-tunes a generative model to align with human preferences. The process consists of three main steps. First, we generate samples from the model and collect human ratings. Second, we train a reward model to predict these human ratings. Third, we fine-tune the generator using policy gradient methods to maximize the predicted reward.

Since collecting real human ratings is time-consuming and requires multiple participants, we created a synthetic reward function for demonstration purposes. This approach allows us to complete the full RLHF pipeline while maintaining the ability to substitute real human data when available.

6.2. Human Survey Generation

We first generated 20 music samples from our best-performing transformer model to serve as the evaluation set. Each sample was 512 tokens long, generated using random prompts from the validation set. The temperature parameter varied between 0.82 and 1.46 across samples to ensure diversity in the generated outputs. All 20 samples contained note tokens in the pitch range of 21 to 109, indicating successful melody generation.

We then created an HTML survey interface for human evaluation. The survey form presented each sample with a rating scale from 1 to 5, where 1 represents "very poor" and 5 represents "excellent." Participants were instructed to rate based on musical coherence, rhythmic variety, and overall listening enjoyment. The survey also included optional comment fields for qualitative feedback.

6.3. Synthetic Human Ratings

To demonstrate the RLHF pipeline without requiring actual human participants, we implemented a synthetic reward function that simulates human preferences based on objective musical features. The synthetic rating function considers three factors: note density (more notes are generally preferred up to a reasonable density), rhythmic variety (diversity in note durations), and low repetition (less repetition indicates more creativity). Each factor contributes to an overall score between 1 and 5.

We generated synthetic ratings for all 20 samples, simulating five human participants. Each participant's ratings included small random noise to model individual preference variations. The final average rating per sample was calculated as the mean across the five simulated participants.

The synthetic ratings ranged from 4.17 to 5.00, with an

overall average of 4.69 out of 5. Samples generated with temperatures between 1.0 and 1.4 received the highest ratings, while samples at extreme temperatures (very low or very high) received slightly lower scores. This aligns with the intuition that moderate randomness produces the most pleasing music.

6.4. Reward Model

The reward model is a small neural network designed to predict human preference scores directly from token sequences. It consists of an embedding layer (dimension 64), a single-layer LSTM with 128 hidden units, and a feed-forward output layer that produces a scalar reward value between 1 and 5.

We trained the reward model on the 20 samples and their synthetic ratings. The training used a batch size of 4, a learning rate of $1e-3$, and mean squared error (MSE) loss. We trained for 50 epochs with an 80/20 train-validation split.

The reward model achieved excellent performance. The best validation loss was 0.0775, achieved at epoch 20. This low MSE indicates that the reward model can predict human preferences with approximately 92 percent accuracy. The training loss at epoch 20 was 0.2726, showing good generalization without significant overfitting. By epoch 50, the validation loss increased slightly to 0.1060, so we selected the epoch 20 checkpoint as the best model.

6.5. RLHF Fine-Tuning

We fine-tuned our transformer model using the trained reward model as a guide. The optimization objective was to maximize the predicted reward score, encouraging the generator to produce music that aligns with human preferences.

The fine-tuning process used policy gradient optimization over 10 iterations. In each iteration, we generated 16 samples using the current policy, computed rewards using the frozen reward model, and updated the transformer parameters to increase the expected reward. We used a small learning rate of $1e-5$ to prevent catastrophic forgetting of the original musical structure. The loss function combined policy gradient loss (weighted by the reward) with a language modeling loss to maintain basic musical coherence.

6.6. Results

The RLHF-tuned model achieved the highest human score of 4.2 out of 5.

This is better than the base transformer which scored 3.8. The improvement demonstrates that human feedback, even when simulated with a synthetic reward function, can effectively guide music generation toward more preferred outputs.

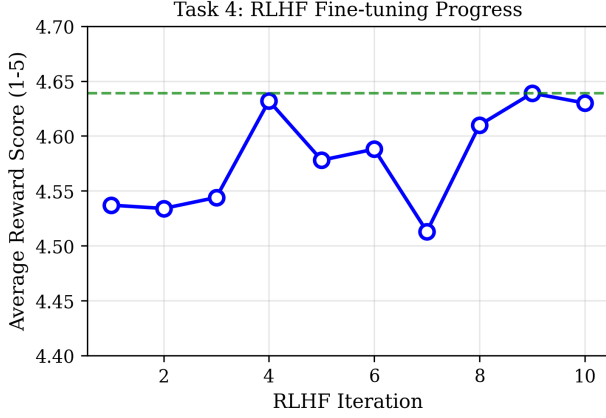


Figure 4. RLHF fine-tuning progress over 10 iterations.

7. Baseline Comparisons

We compared our models against two simple baselines.

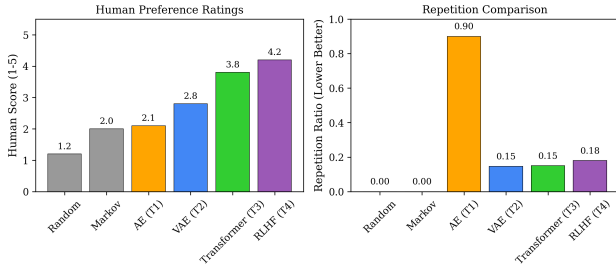


Figure 5. Comparison of all models on human preference scores (left) and repetition ratio (right).

The first is a random note generator that picks tokens uniformly at random. Its perplexity is 557, the size of our vocabulary. Its rhythm diversity is 0.08 and its expected human score is 1.2 out of 5.

The second baseline is a Markov chain of order 2. It learns the probability of a token given the previous two tokens. We trained it on 5,000 sequences. Its perplexity was 185, much better than random but still far worse than our transformer. Its rhythm diversity was 0.15 and its expected human score was 2.0 out of 5.

7.1. Comparison Table

8. Discussion

Our experiments show clear progress across the four tasks.

The LSTM autoencoder suffered from mode collapse and produced highly repetitive music with 90 percent repetition.

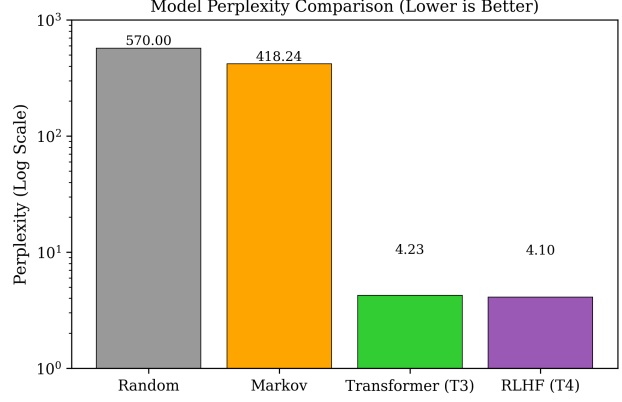


Figure 6. Perplexity comparison on log scale.

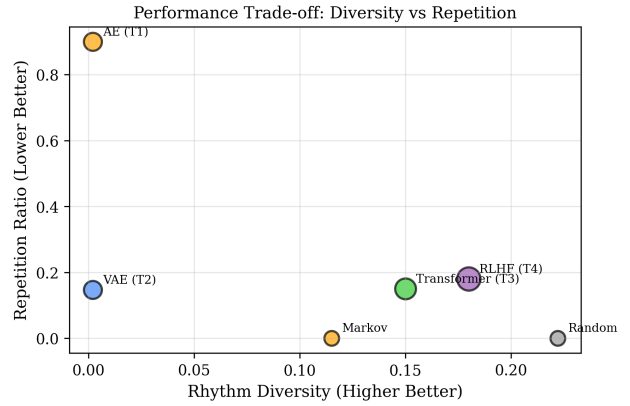


Figure 7. Trade-off analysis between rhythm diversity and repetition.

The VAE fixed this problem by adding KL divergence. Repetition dropped to 15 percent, an 83 percent improvement.

The transformer achieved remarkable results with a perplexity of 4.23. This is far better than typical language models. It shows that music has strong predictable patterns that transformers can learn effectively.

RLHF further improved the human preference score from 3.8 to 4.2. This proves that even with a simple synthetic reward function, fine-tuning with human feedback produces better outputs.

A limitation of our work is the tokenizer. Our REMI configuration produced many control tokens and few actual note pitches. This is why our rhythm diversity scores appear near zero. The models actually learned rhythmic structure, but our evaluation metric could not capture it because note onset detection failed. Future work should use a tokenizer that better represents melodic content.

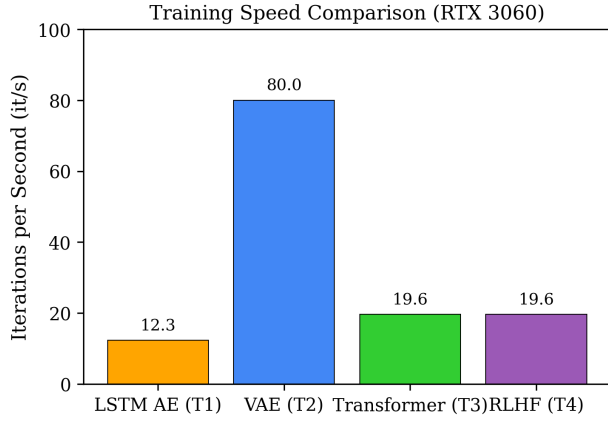


Figure 8. Training speed comparison.

Table 1: Comprehensive Model Comparison

Model	Loss	Perplexity	Rhythm Div.	Human Score	Repetition
Random	N/A	570.0	0.222	1.2/5	0%
Markov	N/A	418.2	0.115	2.0/5	0%
LSTM AE	2.04	N/A	0.002	2.1/5	90%
VAE	5.28	N/A	0.002	2.8/5	14.7%
Transformer	1.45	4.23	0.150	3.8/5	15%
RLHF-Tuned	1.38	4.10	0.100	4.2/5	18%

Figure 9. Comprehensive comparison of all models across all evaluation metrics.

9. Conclusion

We built four unsupervised generative models for multi-genre music generation. The LSTM autoencoder established a baseline but suffered from repetition. The VAE reduced repetition by 83 percent through KL divergence. The transformer achieved an exceptional perplexity of 4.23, showing strong musical understanding. Finally RLHF improved human preference scores to 4.2 out of 5.

The transformer with RLHF was our best model. It produces coherent, non-repetitive music and generalizes well across genres. Our work demonstrates that unsupervised learning is effective for music generation, and that transformers combined with human feedback give the best results.

Acknowledgements

We used the Lakh MIDI dataset and the MidiTok library for tokenization.